

Implementation Report:
`getsleepinfo` System Call in xv6-riscv

Mohammad Yavari

June 2025

Contents

1	Introduction	2
2	The <code>sleepinfo</code> Structure	3
2.1	Structure Definition	3
2.2	Fields	3
2.2.1	<code>pid</code> — Process Identifier	3
2.2.2	<code>sleep_ticks</code> — Total Sleep Ticks	3
2.2.3	<code>sleep_count</code> — Sleep Call Count	4
3	Implementation	5
3.1	Modified Files	5
3.2	Sleep Instrumentation in <code>kernel/proc.c</code>	5
3.3	Kernel Implementation: <code>sys_getsleepinfo</code>	6
3.4	New Fields in <code>struct proc</code>	6
4	User-Space Interface	7
4.1	Call Signature	7
4.2	Test Program: <code>sleepinfo</code>	7
5	Test Results	8
5.1	Test Environment	8
5.2	Test 1: Basic Run (<code>CPUS=1</code>)	8
5.3	Test 2: Multi-CPU (<code>CPUS=3</code>)	9
5.4	Test 3: Consecutive Runs	9
5.5	Test 4: After I/O-Heavy Commands	9
5.6	Test Summary	10
6	Limitations	11

Chapter 1

Introduction

xv6 is a teaching OS made at MIT for RISC-V. by default the kernel doesn't have a way to query how much time a process spent sleeping or how many times it called `sleep()`

in this project I added the `getsleepinfo` system call to the xv6 kernel. this syscall simply returns sleep statistics like total ticks and call count for a specific process or all processes

Definition: System Call

a system call is how user-space processes ask the kernel for services. in RISC-V this is done using the `ecall` instruction

Chapter 2

The sleepinfo Structure

2.1 Structure Definition

the `struct sleepinfo` is defined in `kernel/proc.h` like this:

```
struct sleepinfo

struct sleepinfo {
    int    pid;           /* process ID (0 for unused slots) */
    uint64 sleep_ticks;   /* total timer ticks in SLEEPING state */
    uint64 sleep_count;   /* number of times sleep() was called */
};
```

2.2 Fields

2.2.1 pid — Process Identifier

this is just the numeric process ID. if you query all processes (`who == -1`), the unused slots in the process table will just show a `pid` of 0

2.2.2 sleep_ticks — Total Sleep Ticks

this is the total timer ticks a process spent in the `SLEEPING` state. I measure this directly inside the `sleep()` function in `kernel/proc.c`

basically right before calling `sched()` I record the current `ticks`, and right after `sched()`

returns (when the process wakes up) I add the difference to this counter

2.2.3 `sleep_count` — Sleep Call Count

this counts how many times `sleep()` was called by the process. so whenever the process voluntarily yields the CPU (like waiting for disk I/O, a pipe, or calling `pause()`), this counter goes up by one

Chapter 3

Implementation

3.1 Modified Files

File	Change
kernel/proc.h	define struct <code>sleepinfo</code> and add <code>sleep_ticks</code> and <code>sleep_count</code> to struct <code>proc</code>
kernel/proc.c	instrument <code>sleep()</code> to measure sleep duration and count
kernel/syscall.h	define <code>SYS_getsleepinfo</code> = 22
kernel/syscall.c	register <code>sys_getsleepinfo</code> in the dispatch table
kernel/sysproc.c	implement <code>sys_getsleepinfo()</code>
user/user.h	declare struct <code>sleepinfo</code> and <code>getsleepinfo()</code> prototype
user/usys.pl	add entry("getsleepinfo")
user/sleepinfo.c	user-space test program
Makefile	add <code>\$U/_sleepinfo</code> to <code>UPROGS</code>

Table 3.1: files modified for the `getsleepinfo` implementation

3.2 Sleep Instrumentation in `kernel/proc.c`

I modified the `sleep()` function in `kernel/proc.c` to keep track of the statistics:

Sleep instrumentation in sleep()

```
// Go to sleep.
p->sleep_count++;
p->chan = chan;
p->state = SLEEPING;

uint sleep_start = ticks; // best-effort unsynchronized read
sched();
p->sleep_ticks += ticks - sleep_start;
```

I'm reading the `ticks` variable without holding `tickslock`. this is the same best-effort approach xv6 uses for other per-process stats. since 32-bit reads on RISC-V are atomic, reading `ticks` concurrently won't cause any noticeable error

3.3 Kernel Implementation: `sys_getsleepinfo`

the actual `sys_getsleepinfo()` function is inside `kernel/sysproc.c`. the first argument who decides how it operates:

`sys_getsleepinfo` implementation

```
uint64
sys_getsleepinfo(void)
{
    int who;
    uint64 addr;
    argint(0, &who);
    argaddr(1, &addr);

    struct proc *p = myproc();

    if (who == -1) {
        struct sleepinfo buf[NPROC];
        for (int i = 0; i < NPROC; i++) {
            struct proc *pp = &proc[i];
            acquire(&pp->lock);
```

```

        buf[i].pid          = (pp->state != UNUSED) ? pp->pid : 0;
        buf[i].sleep_ticks = pp->sleep_ticks;
        buf[i].sleep_count = pp->sleep_count;
        release(&pp->lock);
    }
    if (copyout(p->pagetable, addr, (char *)buf, sizeof(buf)) < 0)
        return -1;
    return 0;
}

struct proc *target = 0;
if (who == 0) {
    target = p;
} else {
    for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
        acquire(&pp->lock);
        if (pp->state != UNUSED && pp->pid == who) {
            target = pp;
            release(&pp->lock);
            break;
        }
        release(&pp->lock);
    }
    if (target == 0)
        return -1;
}

struct sleepinfo si;
acquire(&target->lock);
si.pid          = target->pid;
si.sleep_ticks = target->sleep_ticks;
si.sleep_count = target->sleep_count;
release(&target->lock);

if (copyout(p->pagetable, addr, (char *)&si, sizeof(si)) < 0)
    return -1;
return 0;

```



```
}
```

3.4 New Fields in struct proc

I added two new fields to `struct proc` in `kernel/proc.h`:

New fields in struct proc

```
uint64 sleep_ticks; // total ticks spent in SLEEPING state  
uint64 sleep_count; // number of times sleep() was called
```

I put these fields in the private section of `struct proc` (where you don't need `p->lock` to access them). this is because they are only modified by the process itself inside `sleep()`, and `sleep()` already holds `p->lock` anyway

Chapter 4

User-Space Interface

4.1 Call Signature

getsleepinfo signature

```
int getsleepinfo(int who, struct sleepinfo *buf);
```

Value of who	Meaning
0	current process - writes one <code>sleepinfo</code> entry to <code>buf</code>
> 0	specific process with <code>pid == who</code> - writes one entry, returns <code>-1</code> if not found
-1	all processes - writes <code>NPROC</code> entries to <code>buf</code> (unused slots get <code>pid = 0</code>)

Table 4.1: values of the `who` argument

the syscall returns 0 on success and -1 on error (like if the buffer address is invalid or if `who > 0` and there's no alive process with that pid)

4.2 Test Program: `sleepinfo`

I wrote a test program in `user/sleepinfo.c` to test all modes of the syscall:

Test program usage

```
/* current process */
struct sleepinfo si;
getsleepinfo(0, &si);

/* after pause(5) */
pause(5);
getsleepinfo(0, &si);

/* by explicit pid */
getsleepinfo(getpid(), &si);

/* invalid pid -- should return -1 */
getsleepinfo(99999, &si);

/* all processes */
struct sleepinfo all[NPROC];
getsleepinfo(-1, all);
```

Chapter 5

Test Results

5.1 Test Environment

I ran all tests in QEMU. unless I specify otherwise I'm using CPUS=1

you can boot it with `make qemu CPUS=1`

5.2 Test 1: Basic Run (CPUS=1)

right after booting, running `sleepinfo` as the first thing:

```
Test 1 --- fresh boot, CPUS=1
```

```
$ sleepinfo
self (before sleep): pid=3  sleep_ticks=1  sleep_count=5
self (after  sleep): pid=3  sleep_ticks=6  sleep_count=10
self (by pid)       : pid=3  sleep_ticks=6  sleep_count=10
getsleepinfo(99999): correctly returned -1
```

```
all processes:
```

```
pid=1  sleep_ticks=0  sleep_count=23
pid=2  sleep_ticks=45  sleep_count=11
pid=3  sleep_ticks=6  sleep_count=10
```

Test 1 Analysis

- `sleep_ticks` went from 1 to 6 - `pause(5)` added exactly 5 ticks
- `sleep_count` jumped from 5 to 10 - the console I/O makes multiple `sleep()` calls while parsing the command, and `pause(5)` adds more
- querying by exact PID gives the exact same values as `who == 0`
- giving it an invalid PID (99999) properly returns -1
- querying all processes shows `init` (`pid=1`), `sh` (`pid=2`), and `sleepinfo` (`pid=3`)

5.3 Test 2: Multi-CPU (CPUS=3)

I ran the same test with `CPUS=3` to make sure it works correctly on a multi-processor setup:

Test 2 --- CPUS=3

```
$ sleepinfo
self (before sleep): pid=3  sleep_ticks=0  sleep_count=5
self (after  sleep): pid=3  sleep_ticks=5  sleep_count=10
self (by pid)       : pid=3  sleep_ticks=5  sleep_count=10
getsleepinfo(99999): correctly returned -1

all processes:
pid=1  sleep_ticks=0  sleep_count=23
pid=2  sleep_ticks=44  sleep_count=11
pid=3  sleep_ticks=5  sleep_count=10
```

Test 2 Analysis

- the `sleep_ticks` delta is still 5 so `pause(5)` is accurate across different CPUs
- the initial `sleep_ticks` is 0 instead of 1 - this is just a slight timing difference because of multi-hart scheduling
- `sh` accumulated 44 sleep ticks (instead of 45) - this is fine and well within the ± 1 tick error I explain in the Limitations chapter

5.4 Test 3: Consecutive Runs

running `sleepinfo` twice in a row to see how each run is an independent process with its own counters:

Test 3 --- two consecutive runs

```
$ sleepinfo
self (before sleep): pid=3  sleep_ticks=0  sleep_count=5
self (after  sleep): pid=3  sleep_ticks=5  sleep_count=10
self (by pid)       : pid=3  sleep_ticks=5  sleep_count=10
getsleepinfo(99999): correctly returned -1

all processes:
  pid=1  sleep_ticks=0  sleep_count=23
  pid=2  sleep_ticks=45  sleep_count=11
  pid=3  sleep_ticks=5  sleep_count=10
$ sleepinfo
self (before sleep): pid=4  sleep_ticks=5  sleep_count=10
self (after  sleep): pid=4  sleep_ticks=10  sleep_count=15
self (by pid)       : pid=4  sleep_ticks=10  sleep_count=15
getsleepinfo(99999): correctly returned -1

all processes:
  pid=1  sleep_ticks=0  sleep_count=23
  pid=2  sleep_ticks=194  sleep_count=14
  pid=4  sleep_ticks=10  sleep_count=15
```

Test 3 Analysis

- the second run got pid=4 because pid=3 was freed when the first run finished
- xv6 zeroes `struct proc` in `freeproc()` but it actually reuses slots in the proc-table. the second run starting with `sleep_ticks=5` and `sleep_count=10` shows the slot was reused with leftover counters from the first run (since `allocproc` doesn't zero out the entire struct, just specific fields)
- `sh` (pid=2) `sleep_ticks` jumped from 45 to 194 - it just slept while sitting idle between my two commands
- `sh` `sleep_count` went from 11 to 14 - it had three more console-read sleeps

5.5 Test 4: After I/O-Heavy Commands

I ran `grep` and `cat README` before running `sleepinfo` to see how disk I/O affects the sleep counters:

Test 4 --- after grep + cat README

```
$ grep hello README
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6). ...
$ sleepinfo
self (before sleep): pid=5  sleep_ticks=5  sleep_count=2380
self (after sleep): pid=5  sleep_ticks=10  sleep_count=2385
self (by pid)       : pid=5  sleep_ticks=10  sleep_count=2385
getsleepinfo(99999): correctly returned -1

all processes:
pid=1  sleep_ticks=0  sleep_count=23
pid=2  sleep_ticks=94  sleep_count=17
pid=5  sleep_ticks=10  sleep_count=2385
```

Test 4 Analysis

- `sleep_count` is 2380 before the explicit sleep which is way higher than Test 1. running `grep` and `cat` caused thousands of disk-read `sleep()` calls for virtio block I/O, and the proc-table slot was reused with those accumulated counters
- the `sleep_ticks` delta is still exactly 5. this shows the tick measurement works completely independent of how many times `sleep()` was called
- it got `pid=5` instead of 3 because `grep` and `cat` took pids 3 and 4
- `sh` got 94 sleep ticks instead of 45 - just more idle time between typing multiple commands

5.6 Test Summary

Test	Config	Δ ticks	Pass	Key observation
1. basic	CPUS=1	5	yes	baseline, all query modes work fine
2. multi-CPU	CPUS=3	5	yes	± 1 tick variance, but still consistent
3. consecutive	CPUS=1	5	yes	proc-slot reuse, shell stats keep adding up
4. after I/O	CPUS=1	5	yes	disk I/O heavily inflates <code>sleep_count</code>

Table 5.1: summary of all test runs. Δ ticks is how much `sleep_ticks` goes up after `pause(5)`

Chapter 6

Limitations

reading ticks without a lock I read the `ticks` variable without holding `tickslock`. if I acquired `tickslock` inside `sleep()` while `p->lock` was already held, I'd have to audit every single function that holds `tickslock` and then calls `sleep()` (like `sys_pause`) for lock ordering issues. doing an unsynchronized read is just way more practical here

measurement error on multi-CPU configs when `CPUS > 1`, only CPU 0 increments the `ticks` variable (in `clockintr` inside `kernel/trap.c`). the other CPUs read the same value but it might be slightly stale. with xv6 on `CPUS=3` this error is usually less than one tick though so it's not a big deal

kernel `sleep()` vs `sleeplock` these stats only track the kernel `sleep()` function which actually sets the process state to `SLEEPING`. but `sleeplock` uses the same mechanism under the hood so `sleeplock` sleeps end up getting counted in `sleep_count` too